Optional Query Parameters and Data Filtering: Takeaways

by Dataquest Labs, Inc.

Syntax

- Sending a GET request to the World Development Indicators API without any query parameters:

```
import requests
response = requests.get("https://api-server.dataquest.io/economic_data/countries")
data = response.json()
```

- Sending a GET request to the API with a `filter_by` query parameter to refine the data:

```
response = requests.get("https://api-server.dataquest.io/economic_data/countries?
filter_by=region=Sub-Saharan%20Africa")
data = response.json()
```

- Adding multiple query parameters to an API GET request:

```
response = requests.get("https://api-server.dataquest.io/economic_data/indicators?
filter_by=topic=Environment:
%20Agricultural%20production&filter;_by=series_code=AG.AGR.TRAC.NO")
data_str = response.json()
```

- Parsing a JSON response to a Python data structure:

```
import json
data = json.loads(data_str)
```

- Searching within a list of dictionaries for specific data:

```
for indicator in data:
    if indicator['indicator_name'] == "Agricultural machinery, tractors":
        print(f"Indicator Name: {indicator['indicator_name']}")
    break
```

- An example of a GET request with an invalid query parameter:

```
response = requests.get("https://api-server.dataquest.io/economic_data/indicators?
filter_by=indicator_topic=Health: Risk factors")
invalid_data_str = response.json()
print(invalid_data_str)
```

- Implementing a try/except block in API requests:

```
try:
    response = requests.get("https://api-server.dataquest.io/...")
    data = response.json()
except Exception as e:
    print("An error occurred with the request:", e)
```

- Sending a GET request with pagination to fetch a specific segment of data:

```
parameters = {
"limit": 5,
```

```
"offset": 3
}
response = requests.get("https://api-server.dataquest.io/economic_data/indicators",
params=parameters)
data = response.json()
```

- Defining pagination parameters in an API request:

```
parameters = {
"limit": 10,
"offset": 0
}
response = requests.get("https://api-server.dataquest.io/economic_data/indicators",
params=parameters)
data_page_str = response.json()
data_page = json.loads(data_page_str)
```

- Setting up and executing a GET request using optimized pagination:

`parameters = { "limit": 30, "offset": 0 } response = requests.get("https://api-server.dataquest.io/economic_data/indicators", params=parameters) data = json.loads(response.json()) print(data[0])`

- Crafting a GET request with combined query parameters and pagination:

```
parameters = {
"filter_by": "currency_unit=Euro,income_group=High income",
"limit": 5,
"offset": 0
}
response = requests.get("https://api-server.dataquest.io/economic_data/countries",
params=parameters)
data = json.loads(response.json())
for record in data:
    print(record.get("country_code", []))
```

- Setting up a paginated API request with

```
page
```

and

```
page_number
```

:

`parameters = { "page": 1, "page_number": 50 } response = requests.get("https://api-server.dataquest.io/economic_data/historical_data", params=parameters) data = response.json()`

Concepts

- **Optional Query Parameters and Filters**: Customize API data retrieval like choosing pizza toppings, using filters to refine searches like using specific sieves for precision.
- **URL Encoding and JSON Parsing**: URL encoding ensures accurate interpretation of special characters, akin to a secret code. JSON parsing translates data into a Python-friendly format, much like translating a foreign language.

- **Handling API Errors and Documentation**: Error messages are clues for troubleshooting, similar to solving a detective case. API documentation is the essential guide, akin to a cookbook for API queries.
- **Error Handling Techniques**: Using try/except in Python adds safety nets to code, preventing crashes. Interpreting error messages is key for issue resolution, like diagnosing car problems based on dashboard indicators.
- **Pagination and Data Retrieval Efficiency**: Managing data like reading a book one chapter at a time. Selecting the right limit and offset for data retrieval is akin to planning an efficient road trip.
- **Combined Strategies for Data Management**: Filtering and pagination together optimize data retrieval, like a librarian organizing books. Page-based pagination enhances user experience, similar to a book's table of contents.

Resources

- API Query Parameters and Filters
- Understanding URL Encoding and JSON Parsing